

ORA-12570 TNS Packet Reader Failure Analysis

Date: 2026-01-29

Issue: ORA-12570: TNS:packet reader failure in GCP setup with 35 concurrent bordero/rollcart functions

Executive Summary

Problem: ORA-12570 "TNS:packet reader failure" errors are occurring in our GCP Cloud Functions deployment with 35 branches, each running bordero and rollcart functions that connect to separate Oracle schemas.

Root Cause:

1. **No connection pooling configuration** - Connection strings lack explicit pooling parameters, leading to uncontrolled connection reuse
2. **GCP network timeouts** - Load balancers drop idle TCP connections after 10 minutes
3. **No connection validation** - Stale/dead pooled connections are returned without health checks
4. **Cloud Function lifecycle mismatch** - Container reuse across invocations attempts to reuse connections that have been dropped by GCP infrastructure

Solution:

- Add connection pooling parameters to all Oracle connection strings (Max Pool Size=10 , Connection Lifetime=300 , Validate Connection=true)
- Configure `SQLNET.EXPIRE_TIME=5` to send keepalive probes
- Force connection recycling every 5 minutes (before GCP's 10-minute timeout)

Expected Impact: Near-elimination of ORA-12570 errors, better resource utilization, improved reliability under concurrent load.

Understanding Cloud Functions Connection Lifecycle

How GCP Cloud Functions Work

Cloud Functions operate differently from traditional always-running servers, which is critical to understanding the Oracle connection issue:

1. Cold Start vs Warm Start

Cold Start (First Invocation):

User Request → GCP spins up new container → Initialize runtime → Load dependencies → Execute global scope → Create connection pool → Execute function → Keep container alive

Warm Start (Subsequent Invocation):

User Request → Reuse existing container → Skip initialization → Reuse existing connection pool → Execute function

2. Container Lifecycle Phases

Phase 1: Initialization (Cold Start Only)

- GCP allocates a new container/sandbox
- .NET runtime is initialized
- Application assemblies are loaded
- **Global scope code executes** (service registration, static initializers)
- Connection pools are created at this stage

Phase 2: Invocation

- HTTP request arrives
- Function handler is called
- DbContext is created from factory
- Connection is requested from the pool
- Database operations execute
- DbContext is disposed
- Connection returns to pool (NOT closed)
- Response is sent

Phase 3: Idle State

- Container remains alive for potential reuse
- Connection pools stay in memory
- TCP connections remain open (but idle)
- **Critical:** GCP infrastructure may drop idle TCP connections after 10 minutes
- Oracle may also close connections based on server timeout settings

Phase 4: Container Reuse or Termination

- If new request arrives within ~15 minutes: Container is reused (warm start)
- If no requests: Container is terminated, connections are lost
- **Problem:** Connection pool is unaware that underlying TCP connections were dropped

How Our Application Creates Connections

Our code creates connections through this flow:

1. Function Invocation (BorderoUploadFunction.cs , RollkartUploadFunction.cs)

```
public async Task<IActionResult> Handle(HttpRequest req)
{
    // Function executes
    var result = await _service.ProcessData(company, branch);
    // DbContext is disposed here, connection returns to pool
}
```

2. Service Layer (DigiLiSService.cs)

```
public async Task ProcessData(int company, int branch)
{
    var context = await _contextProvider.GetVendorContextAsync(company, branch);
    // Use context
} // Context disposed, connection returned to pool
```

3. Context Provider (DigiLiSDbContextProvider.cs)

```
public async Task<DigiLiSdbContext> GetVendorContextAsync(int company, int branch)
{
    var connectionString = await GetConnectionStringFromSecretManager(...);
    return _factory.CreateVendorContext(connectionString);
}
```

4. Factory Creates Connection (DigiLiSdbContextFactory.cs:31-44)

```
private DbContextOptions<DigiLiSdbContext> GetVendorContextOptions(string connectionString)
{
    var optionsBuilder = new DbContextOptionsBuilder<DigiLiSdbContext>();
    var oracleConnection = new OracleConnection(connectionString);
    // No pooling parameters in connectionString!
    return optionsBuilder.UseOracle(oracleConnection).Options;
}
```

Connection Pool Behavior in Cloud Functions

What Happens in a Cold Start:

1. Container starts
2. First OracleConnection created with connection string "User Id=digilis1034;..."
3. Oracle.ManagedDataAccess.Core creates a NEW connection pool for this connection string
4. Min Pool Size connections are established (default: 0, so none)
5. Function requests a connection
6. Pool creates 1 new connection to Oracle
7. Function completes, connection returns to pool
8. Connection stays in pool (NOT closed)

What Happens in Warm Starts:

9. Second request arrives (warm start)
10. Same connection pool exists from step 3
11. Function requests connection
12. Pool returns the SAME connection from step 6 (if still valid)
13. Function executes, returns connection to pool

What Happens After 10+ Minutes Idle:

14. Container is idle for 10 minutes
15. GCP load balancer drops the idle TCP connection
16. Connection pool is UNAWARE the TCP connection was dropped
17. New request arrives
18. Pool returns the "dead" connection
19. Application tries to use it → ORA-12570: TNS:packet reader failure
20. Retry logic kicks in, new connection is created
21. New connection works, returns to pool

The Critical Problem

Connection Pool Assumptions:

- Connection pools are designed for long-running servers
- They assume TCP connections stay alive indefinitely
- They cache connections for performance

Cloud Functions Reality:

- Containers are ephemeral but may live for hours
- GCP infrastructure drops idle connections after 10 minutes
- Oracle may drop connections based on server settings
- Pool has no way to know the underlying TCP socket is dead

Why This Causes ORA-12570:

1. Application creates OracleConnection with no pooling parameters
2. Default pooling behavior: connections never expire (Connection Lifetime=0 means infinite)
3. Connection is used, returned to pool, TCP socket stays open
4. 10+ minutes pass, GCP drops the TCP connection
5. Next invocation gets the "cached" connection from pool
6. Pool thinks connection is valid (it was never disposed)
7. Application tries to send Oracle TNS packets
8. Packet reader fails because TCP socket is closed → ORA-12570

Why This Affects Multiple Branches

Concurrent Load Multiplier Effect:

35 branches × 2 functions (bordero + rollcart) = 70 potential concurrent functions
Each function = separate container with separate connection pool
Each connection string (DIGILIS-{company}-{branch}) = separate pool

Result: 35+ different connection pools, each caching stale connections

Sequential Execution Per Branch:

- Bordero function runs first (may take several minutes)
- Rollcart function runs after bordero completes
- Both reuse the same connection pool within the container
- If bordero finishes, container is idle until rollcart starts
- Connection becomes stale during this gap

Current Connection String Format

User Id=digilis1034;Password=mockPassword;Data Source=digilis.1034:1521/digilis.1034;

Missing Parameters:

- No Connection Lifetime → connections never recycled
- No Validate Connection → no health check before use
- No Max Pool Size → unlimited connections possible
- No explicit Pooling=true → relying on default behavior

Problem Overview

We're experiencing ORA-12570 errors in our GCP Cloud Functions deployment where 35 branches each spawn 2 functions (bordero and rollcart) in series, creating significant concurrent load on Oracle databases.

Current Connection Handling

Connection Factory Implementation

File: DigiLiSDbContextFactory.cs:31-44

```
private DbContextOptions<DigiLiSDbContext> GetVendorContextOptions(string connectionString)
{
    var optionsBuilder = new DbContextOptionsBuilder<DigiLiSDbContext>();
    var oracleConnection = new OracleConnection(connectionString);

    return optionsBuilder.UseOracle(oracleConnection).Options;
}
```

Current Connection String Format

User Id=digilis1034;Password=mockPassword;Data Source=digilis.1034:1521/digilis.1034;

Critical Finding: No connection pooling parameters are configured.

Root Cause Analysis

1. No Connection Pooling Configuration

- Oracle.ManagedDataAccess.Core v23.9.1 uses connection pooling by default
- Without explicit limits, it can create too many connections or reuse stale connections
- With 35 branches × 2 functions = 70 concurrent connections, this overwhelms the Oracle server

2. GCP Network Timeouts

- GCP load balancers drop idle TCP connections after **10 minutes** (600 seconds)
- ORA-12570 "packet reader failure" occurs when trying to reuse pooled connections that have been dropped by GCP infrastructure
- No keepalive mechanism configured to prevent connection drops

3. No Connection Validation

- Pooled connections aren't validated before use
- Dead/stale connections get returned from the pool, causing immediate failures

4. Cloud Function Container Reuse

- Containers are reused across invocations (warm starts)
- Connection pools persist in memory between invocations

- Pools attempt to reuse connections that were dropped by GCP during idle periods
- No mechanism to detect or refresh stale connections

Recommended Solution

Oracle Connection String Parameters

Add these parameters to all connection strings in Google Secret Manager:

```
User Id=digilis1034;  
Password=mockPassword;  
Data Source=digilis.1034:1521/digilis.1034;  
Pooling=true;  
Min Pool Size=1;  
Max Pool Size=10;  
Connection Lifetime=300;  
Connection Timeout=60;  
Validate Connection=true;  
Incr Pool Size=2;  
Decr Pool Size=1;
```

Parameter Explanations

Parameter	Value	Purpose
Pooling	true	Explicitly enable connection pooling
Min Pool Size	1	Minimum connections to maintain per pool
Max Pool Size	10	Limit maximum connections per connection string
Connection Lifetime	300	Force reconnection every 5 minutes (below GCP's 10-min timeout)
Connection Timeout	60	Timeout for establishing new connections (seconds)
Validate Connection	true	Test connections before returning from pool
Incr Pool Size	2	Add 2 connections when pool exhausted

Parameter	Value	Purpose
Decr Pool Size	1	Remove 1 connection when pool shrinks

Key Design Decisions

1. Connection Lifetime = 300 seconds (5 minutes)

- Below GCP's 10-minute idle timeout
- Ensures connections are recycled before network drops them
- Prevents stale connection reuse
- Works with Cloud Function container reuse pattern

2. Max Pool Size = 10

- Limits connections per connection string
- With 35+ different schemas, this prevents overwhelming Oracle
- Oracle recommendation: 1-10 connections per CPU core
- Each function typically needs 1-2 concurrent connections

3. Validate Connection = true

- Tests connection health before returning from pool
- Catches dead connections early
- Small performance overhead but critical for reliability
- Essential for detecting GCP-dropped connections

Additional Configuration: SQLNET.EXPIRE_TIME

Oracle's `SQLNET.EXPIRE_TIME` parameter sends keepalive probes to prevent firewall/load balancer timeouts.

Option 1: Programmatic Configuration

Add to `DigiLiSdbContextFactory` or startup:

```
// During application startup
OracleConfiguration.SqlNetExpireTime = 5; // Send keepalive every 5 minutes
```

Option 2: Via Connection String

```
...;Statement Cache Size=10;Load Balancing=true;
```

Note: Oracle.ManagedDataAccess.Core may have limited support for sqlnet.ora file-based configuration. Programmatic configuration is preferred.

Implementation Steps

Step 1: Update Secret Manager Connection Strings

For each of the 35+ database connection strings in Google Secret Manager (format: DIGILIS-{company}-{branch}):

1. Retrieve current secret value
2. Append pooling parameters
3. Update secret with new version

Example script:

```
# Get current secret
gcloud secrets versions access latest --secret="DIGILIS-10-52"

# Update with new connection string including pooling parameters
echo "User Id=digilis1052;Password=xxx;Data Source=...;Pooling=true;Min Pool Size=1;Max
gcloud secrets versions add DIGILIS-10-52 --data-file=-
```

Step 2: Add SQLNET.EXPIRE_TIME Configuration

Update DigiLiSSetupExtensions.cs :

```
public static IServiceCollection AddDigiLiS(this IServiceCollection services)
{
    // Configure Oracle keepalive
    OracleConfiguration.SqlNetExpireTime = 5; // 5 minutes

    services.AddScoped<IDigiLiSDbContextFactory, DigiLiSDbContextFactory>();
    services.AddScoped<IDigiLiSDbContextProvider, DigiLiSDbContextProvider>();
    services.AddScoped<IDigiLiSService, DigiLiSService>();

    return services;
}
```

Step 3: Add Connection Pool Monitoring (Optional)

Update `DigiLiSdbContextFactory.cs` to log pool statistics:

```
private DbContextOptions<DigiLiSdbContext> GetVendorContextOptions(string connectionString)
{
    var optionsBuilder = new DbContextOptionsBuilder<DigiLiSdbContext>();
    var oracleConnection = new OracleConnection(connectionString);

    // Optional: Log for debugging
    logger.LogDebug("Creating Oracle connection for schema {Schema}",
        connectionString.Contains("digilis") ? "digilis..." : "unknown");

    return optionsBuilder.UseOracle(oracleConnection).Options;
}
```

Step 4: Testing

1. Deploy updated configuration to a single branch first
2. Monitor for ORA-12570 errors
3. Check Cloud Logging for connection pool behavior
4. Gradually roll out to remaining branches

Alternative: Database Resident Connection Pooling (DRCP)

If you control the Oracle database server, consider enabling DRCP:

Benefits

- Handles tens of thousands of concurrent connections
- Server-side connection pooling
- Better resource utilization

Configuration

On Oracle database:

```
-- Start connection pool
EXEC DBMS_CONNECTION_POOL.START_POOL();

-- Configure pool
EXEC DBMS_CONNECTION_POOL.ALTER_PARAM('', 'MINSIZE', '5');
EXEC DBMS_CONNECTION_POOL.ALTER_PARAM('', 'MAXSIZE', '100');
EXEC DBMS_CONNECTION_POOL.ALTER_PARAM('', 'INCRSIZE', '5');
```

Update connection string:

```
...;Data Source=digilis.1034:1521/digilis.1034:POOLED;
```

Note: Requires Oracle Database 11g or higher.

Expected Outcomes

After implementing these changes:

1. **Reduced ORA-12570 errors:** Connection validation prevents stale connection usage
2. **Better resource utilization:** Connection pooling limits prevent overwhelming Oracle
3. **Automatic connection refresh:** Connection Lifetime ensures fresh connections
4. **Network resilience:** Keepalive probes prevent timeout-based drops
5. **Cloud Function compatibility:** Connection recycling works with container reuse pattern

Monitoring

Monitor these metrics post-deployment:

- ORA-12570 error frequency (should approach zero)
- Connection pool statistics (if logging enabled)
- Oracle database session counts
- Cloud Function cold start latency (may increase slightly with validation)
- Overall function success rate

References

- [Simulating KeepAlive in Oracle with SQLNET.EXPIRE_TIME](#)

- [Database Resident Connection Pool \(DRCP\) in Oracle Database](#)
- [Configuring Database Resident Connection Pooling](#)
- [Session Pooling and Connection Pooling in OCI](#)
- [ODP.NET – "Pooling" and "Connection request timed out"](#)
- [GCP Connection Timeout Documentation](#)
- [Oracle Documentation - ORA-12570](#)

Technical Context

Affected Components

- **Functions:** BorderoUploadFunction, RollkartUploadFunction
- **Services:** DigiLiSService
- **Infrastructure:** DigiLiSDBContextFactory, DigiLiSDBContextProvider
- **Dependencies:** Oracle.ManagedDataAccess.Core v23.9.1, Oracle.EntityFrameworkCore v8.23.60

Connection Architecture

```

Cloud Functions (35 branches × 2 functions)
    ↓
DigiLiSDBContextFactory
    ↓
Google Secret Manager (retrieve connection string)
    ↓
OracleConnection (with pooling parameters)
    ↓
Oracle Database (per schema: DIGILIS{company}{branch})
  
```

Existing Retry Logic

The codebase already has retry logic for Oracle operations:

- Max retry count: **5 attempts**
- Exponential backoff: `100 * Math.Pow(2, retryAttempt - 1)` milliseconds
- Catches `DbException`

This retry logic will work better with connection validation enabled, as it can quickly retry after detecting a dead connection.

Summary and Conclusion

The Problem in Simple Terms

Our Cloud Functions are trying to reuse database connections that were silently dropped by Google's network infrastructure after 10 minutes of inactivity. The connection pool doesn't know these connections are dead, so it hands them back to our code, which immediately fails with ORA-12570 errors.

Why This Happens Specifically in Cloud Functions

Unlike traditional servers that run continuously, Cloud Functions:

- Spin up on demand and may stay alive for hours
- Reuse containers and connection pools across multiple invocations
- Experience unpredictable idle periods between invocations
- Are subject to GCP infrastructure timeouts that don't exist in traditional hosting

This creates a perfect storm where connection pools designed for always-running servers encounter network timeouts designed for stateless serverless functions.

The Fix

Two simple configuration changes eliminate the problem:

1. **Connection Lifetime = 300 seconds:** Forces new connections every 5 minutes, before GCP can drop them
2. **Validate Connection = true:** Checks connection health before use, catches any that slip through

Implementation Priority

Critical (Do immediately):

- Add `Connection Lifetime=300` to all connection strings
- Add `Validate Connection=true` to all connection strings
- Add `Max Pool Size=10` to prevent connection explosion

Important (Do soon):

- Add `OracleConfiguration.SqlNetExpireTime = 5` to keep connections alive
- Add logging to monitor connection pool behavior

Optional (Nice to have):

- Consider DRCP for server-side pooling if you control the database
- Add metrics/dashboards for ongoing monitoring

Expected Timeline

- **Configuration change:** 1-2 hours to update all 35+ secrets in Secret Manager
- **Code change:** 30 minutes to add `SqlNetExpireTime` configuration
- **Testing:** Deploy to 1 branch, monitor for 24 hours
- **Full rollout:** Gradual deployment over 1 week with monitoring

Success Criteria

- ORA-12570 error rate drops to near zero ($< 0.1\%$ of requests)
- Oracle database connection count remains stable and predictable
- Function success rate improves to $> 99.9\%$
- No increase in function execution time or cold start latency

Risk Assessment

Low Risk Changes:

- Adding pooling parameters only affects how connections are managed
- No code changes required, only configuration
- Can be rolled back instantly by reverting Secret Manager versions
- Changes are well-documented Oracle best practices

Potential Issues:

- Slight increase in connection creation overhead (negligible)
- Small validation overhead per connection use ($\sim 10\text{ms}$)
- May expose other latent database issues if connections were hiding problems

Next Steps

1. **Immediate:** Update connection strings in Secret Manager with pooling parameters
2. **Day 1:** Deploy to single test branch, monitor for 24 hours

3. **Day 2:** If successful, deploy to 5 branches
4. **Week 1:** Gradual rollout to all 35 branches
5. **Week 2:** Add SqlNetExpireTime configuration as code change
6. **Week 3:** Review metrics and adjust pool sizes if needed

This solution addresses the fundamental mismatch between Cloud Functions' ephemeral nature and Oracle's connection pooling expectations, resulting in reliable database connectivity regardless of function invocation patterns.